DEPARTMENT OF COMPUTER SCIENCE AND ENGINEERING

U23IT593-DEVOPS RECORD

SRI ESHWAR COLLEGE OF ENGINEERING

KINATHUKADAVU,

COIMBATORE – 641202

SRI ESHWAR COLLEGE OF ENGINEERING

KINATHUKADAVU, COIMBATORE-641202

(An Autonomous Institution Affiliated to Anna University, Chennai)

DEPARTMENT OF COMPUTER SCIENCE AND ENGINEERING

BONAFIDE CERTIFICATE

Certified that this is the bonafide record of work done by Mr. /Ms.…….

……Register No: ..... of III Year **B.E**

…......in the **U23IT593-Devops** during the 5th **Semester** of the academic year **2026 – 27 (Odd Semester).**

Signature of Staff In-charge **Head of the department**

Submitted for the End Semester Practical Examinations, held on…….

Internal Examiner **External Examiner**

EX. NO.	DATE	NAME OF THE EXPERIMENTS	PAGE NO.	MARKS (75)	SIGN
1		Install and configure Git for version control on Linux or Windows System			
2		Push a local repository to GitHub and manage remote synchronization using Git push and pull			
3		Create a Jenkins Freestyle job to clone a GitHub repository			
4		Create a simple web application using the Maven webapp archetype			
5		Use the Jenkins pipeline to compile and test a Java project using Maven.			
6		Set up Jenkins credentials to securely access private GitHub repositories during CI builds.			
7		Provision an EC2 instance with a security group and elastic IP using Terraform			
8		Create a playbook to manage users and groups with custom SSH key distribution			
9		Deploy Grafana and integrate it with Prometheus as a data source to visualize metrics			
10		Install and configure the ELK stack to collect and visualize application logs.			

Signature: **Average:**

Ex.No : 01 Date:	**Install and configure Git for version control on a Linux or Windows system.**

Aim

To install Git on a Linux or Windows system and configure it with basic user information for version control operations.

Requirements

- Windows 10/11 or Ubuntu Linux
- Internet Connection
- Git Software
- Administrator (Windows) or sudo (Linux) privileges

Procedure

Installation on Windows:

Step 1: Download Git

- Open a web browser.
- Visit Git from the official website: https://git-scm.com/downloads
- Download the latest version for Windows.

Step 2: Install Git

- Double-click the downloaded installer.
- Keep the default installation settings.
- Click **Next** until the installation completes.
- Click **Finish**.

Step 3: Verify Installation

Open **Command Prompt** or **Git Bash** and execute:
git –version

Step 4: Configure Git Username

git config --global user.name "Your Name"

Example:
git config --global user.name "Harshitha"

Step 5: Configure Email

git config --global user.email harshitha@example.com

Step 6: Verify Configuration

git config --list
(Or)
git config --global --list

Output

```
PS C:\Users\91904> git --version
git version 2.55.0.windows.2
```

```
PS C:\Users\91904> git config --global user.name "Harshitha"
PS C:\Users\91904> git config --global user.email "harshitha@gmail.com"
```

```
PS C:\Users\91904> git config --list
diff.astextplain.textconv=astextplain
filter.lfs.clean=git-lfs clean -- %f
filter.lfs.smudge=git-lfs smudge -- %f
filter.lfs.process=git-lfs filter-process
filter.lfs.required=true
http.sslbackend=schannel
core.autocrlf=true
core.fscache=true
core.symlinks=false
pull.rebase=false
credential.helper=manager
credential.https://dev.azure.com.usehttppath=true
init.defaultbranch=master
user.name=Harshitha
user.email=harshitha@gmail.com
```

```
PS C:\Users\91904> git config --global --list
user.name=Harshitha
user.email=harshitha@gmail.com
```

SRI ESHWAR COLLEGE OF ENGINEERING DEPARTMENT OF CSE		
PARAMETERS	**MAX MARKS**	**MARKS OBTAINED**
Pre Lab	20	
Program	25	
Output	20	
Viva	10	
Total	75	
SIGNATURE OF THE FACULTY		

Result

Git was successfully installed and configured on the Linux/Windows system. The installation was verified using the git --version command, and the global username and email were configured successfully, enabling the system to use Git for version control.

Ex.No : 02	Push a local repository to GitHub and manage remote synchronization using Git push and pull.
Date:	

Aim

To create a local Git repository, connect it to a GitHub remote repository, push local changes to GitHub, and synchronize updates between the local and remote repositories using git pushand git pull.

Requirements

- Git installed on the system
- GitHub account
- A GitHub repository created

Procedure

Step 1: Create a Local Repository

Open Git Bash or Command Prompt and execute:

```
mkdir GitDemo
cd GitDemo
```

Step 2: Initialize a Git Repository

```
git init
```

Step 3: Create a Sample File

```
echo "Welcome to DevOps Lab" > demo.txt
```

Step 4: Check Repository Status

```
git status
```

Step 5: Add the File to the Staging Area

```
git add demo.txt
```

Step 6: Commit the File

```
git commit -m "Initial Commit"
```

Step 7: Connect the Local Repository to GitHub

```
git remote add origin https://github.com/username/GitDemo.git
```

Verify the remote:

```
git remote -v
```

Step 8: Push the Repository to GitHub

```
git branch -M main
git push -u origin main
```

Step 9: Verify the Repository on GitHub

Refresh the GitHub repository page.
Confirm that the README.md file has been uploaded.

Step 10: Make Changes Locally

echo "Git Push and Pull Demo" >> demo.txt

Step 11: Commit the Changes

git add demo.txt

git commit -m "Updated txt file"

Step 12: Push the Updated Changes

git push origin main

Step 13: Pull Changes from GitHub

If changes are made in GitHub by another user or through the web interface:

git pull origin main

Output

```
PS C:\Users\91904> cd GitDemo1
PS C:\Users\91904\GitDemo1> git init
Initialized empty Git repository in C:/Users/91904/GitDemo1/.git/
PS C:\Users\91904\GitDemo1> echo "Welcome to DevOps Lab" > demo.txt
```

```
PS C:\Users\91904\GitDemo> git status
On branch master

No commits yet

Untracked files:
  (use "git add <file>..." to include in what will be committed)
        demo.txt

nothing added to commit but untracked files present (use "git add" to track)
```

```
PS C:\Users\91904\GitDemo> git add demo.txt
PS C:\Users\91904\GitDemo> git commit -m "Initial Commit"
[master (root-commit) 903610d] Initial Commit
 1 file changed, 0 insertions(+), 0 deletions(-)
 create mode 100644 demo.txt
```

```
PS C:\Users\91904\GitDemo1> git remote add origin https://github.com/Harshitha-06/Gitbasics2.git
PS C:\Users\91904\GitDemo1> git remote -v
origin  https://github.com/Harshitha-06/Gitbasics2.git (fetch)
origin  https://github.com/Harshitha-06/Gitbasics2.git (push)
```

```
PS C:\Users\91904\GitDemo1> git branch -M main
PS C:\Users\91904\GitDemo1> git push -u origin main
Enumerating objects: 3, done.
Counting objects: 100% (3/3), done.
Writing objects: 100% (3/3), 255 bytes | 23.00 KiB/s, done.
Total 3 (delta 0), reused 0 (delta 0), pack-reused 0 (from 0)
To https://github.com/Harshitha-06/Gitbasics2.git
 * [new branch]      main -> main
branch 'main' set up to track 'origin/main'.
```

```
PS C:\Users\91904\GitDemo1> echo "Git Push and Pull Demo" > demo.txt
PS C:\Users\91904\GitDemo1> git add demo.txt
PS C:\Users\91904\GitDemo1> git commit -m "Updated txt file"
[main fa71d48] Updated txt file
 1 file changed, 0 insertions(+), 0 deletions(-)
```

```
PS C:\Users\91904\GitDemo1> git push origin main
Enumerating objects: 5, done.
Counting objects: 100% (5/5), done.
Writing objects: 100% (3/3), 243 bytes | 121.00 KiB/s, done.
Total 3 (delta 0), reused 0 (delta 0), pack-reused 0 (from 0)
To https://github.com/Harshitha-06/Gitbasics2.git
   c80010c..4408db9  main -> main
```

```
PS C:\Users\91904\GitDemo1> git pull origin main
From https://github.com/Harshitha-06/Gitbasics2
 * branch            main       -> FETCH_HEAD
Already up to date.
```

SRI ESHWAR COLLEGE OF ENGINEERING DEPARTMENT OF CSE		
PARAMETERS	**MAX MARKS**	**MARKS OBTAINED**
Pre Lab	20	
Program	25	
Output	20	
Viva	10	
Total	75	
SIGNATURE OF THE FACULTY		

Result

The local Git repository was successfully connected to the remote GitHub repository. Changes were uploaded to GitHub using git push, and remote updates were synchronized to the local repository using git pull. Thus, remote repository management and synchronization were performed successfully using Git.

Ex.No: 03	Create a Jenkins Freestyle job to clone a GitHub repository
Date:	

Aim

To create a Jenkins Freestyle Job that clones a GitHub repository.

Prerequisites

- Jenkins installed and running
- Git installed on the Jenkins machine
- A GitHub repository URL

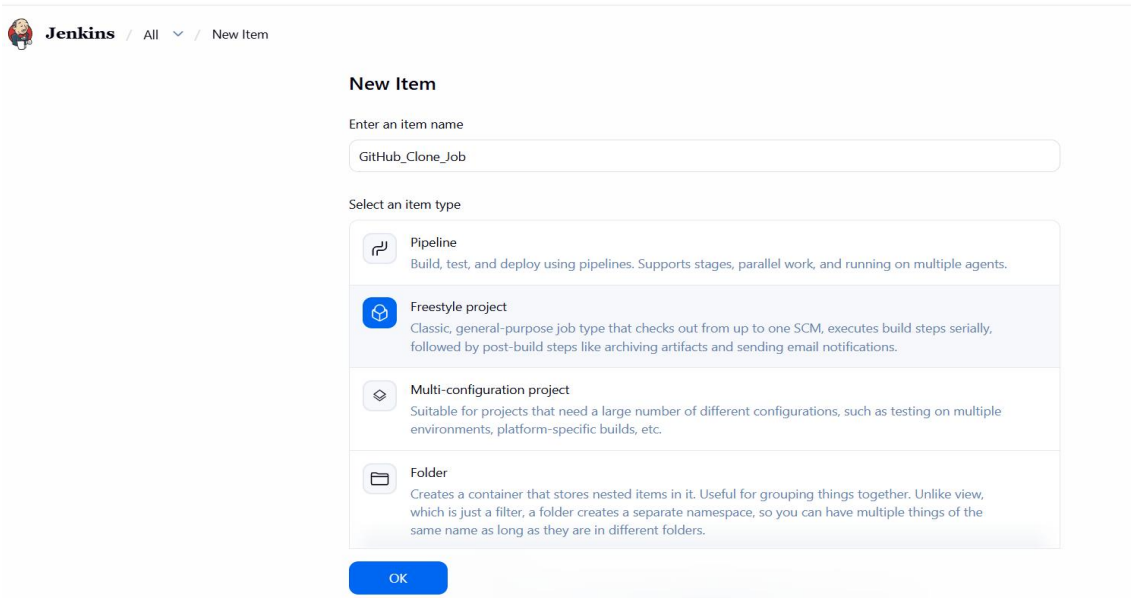
Procedure

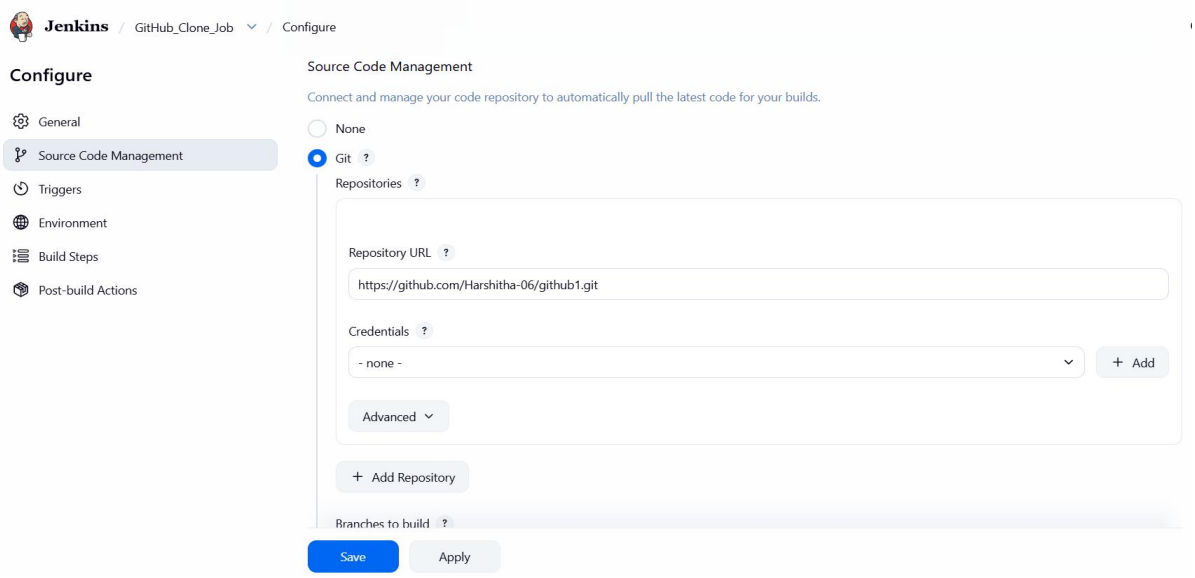
1. Open Jenkins in a web browser http://localhost:8080
2. Click **New Item**.
3. Enter a job name (e.g., GitHub_Clone_Job).
4. Select **Freestyle Project** and click **OK**.
5. In the **Source Code Management** section:
   - Select **Git**.
   - Enter the GitHub repository URL in the **Repository URL** field.
     Example:

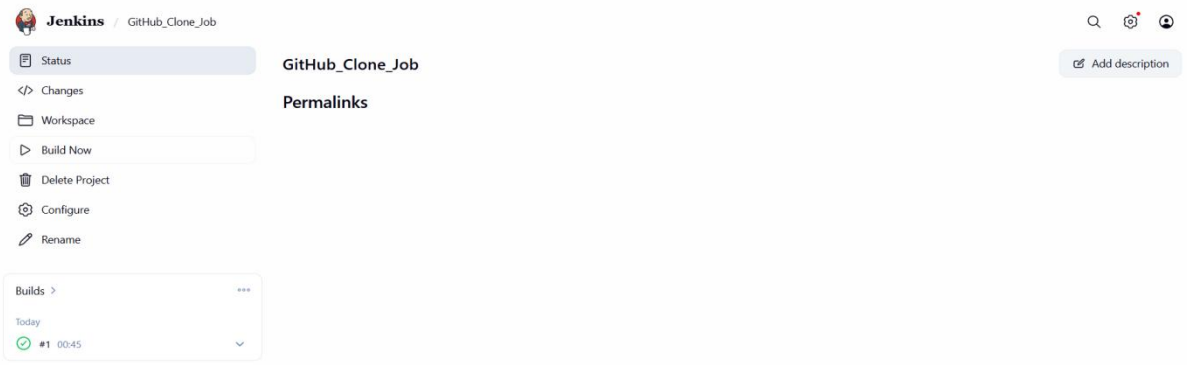
     https://github.com/username/repository.git
6. If the repository is private:
   - Click **Add Credentials**.
   - Enter your GitHub username and Personal Access Token (PAT).
7. In the **Branches to build** field, enter:

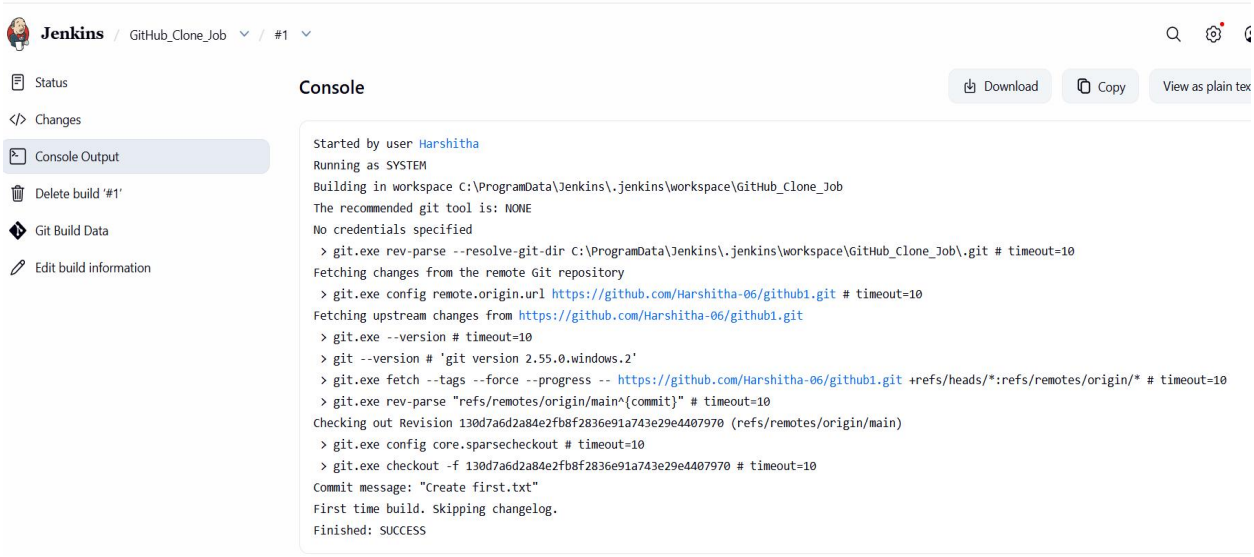
   */main
8. Click **Save**.
9. Click **Build Now** from the left menu.
10. After the build completes, click the build number and select **Console Output** to verify that the repository was cloned successfully.

Output


Jenkins / All / New Item
New Item
Enter an item name
GitHub_Clone_Job
Select an item type
Pipeline
Build, test, and deploy using pipelines. Supports stages, parallel work, and running on multiple agents.
Freestyle project
Classic, general-purpose job type that checks out from up to one SCM, executes build steps serially, followed by post-build steps like archiving artifacts and sending email notifications.
Multi-configuration project
Suitable for projects that need a large number of different configurations, such as testing on multiple environments, platform-specific builds, etc.
Folder
Creates a container that stores nested items in it. Useful for grouping things together. Unlike view, which is just a filter, a folder creates a separate namespace, so you can have multiple things of the same name as long as they are in different folders.
OK


Jenkins / GitHub_Clone_Job / Configure
Configure
General
Source Code Management
Triggers
Environment
Build Steps
Post-build Actions
Source Code Management
Connect and manage your code repository to automatically pull the latest code for your builds.
None
Git
Repositories
Repository URL
https://github.com/Harshitha-06/github1.git
Credentials
- none -
Add
Advanced
Add Repository
Branches to build
Save
Apply


Jenkins / GitHub_Clone_Job
Status
Changes
Workspace
Build Now
Delete Project
Configure
Rename
Builds
Today
#1 00:45
GitHub_Clone_Job
Add description
Permalinks



SRI ESHWAR COLLEGE OF ENGINEERING DEPARTMENT OF CSE		
PARAMETERS	**MAX MARKS**	**MARKS OBTAINED**
Pre Lab	20	
Program	25	
Output	20	
Viva	10	
Total	75	
SIGNATURE OF THE FACULTY		

Result

The Jenkins Freestyle job was created successfully, and the GitHub repository was cloned into the Jenkins workspace. The build completed successfully with a SUCCESS status.

Ex.No : 04	Create a simple web application using the Maven webapp archetype.
Date:	

Aim

To create a simple Java Web Application using the Maven WebApp Archetype, package it as a WAR file, and deploy it on Apache Tomcat Server.

Prerequisites

- Java JDK Installed
- Apache Maven Installed
- Apache Tomcat Server Installed

Procedure

Step 1: Verify Maven Installation
Command: **mvn --version**

Step 2: Create Maven Web Application
Open Command Prompt and move to the location where you want to create the project.

Execute:

```
mvn archetype:generate ^
-DgroupId=com.example ^
-DartifactId=SimpleWebApp ^
-DarchetypeArtifactId=maven-archetype-webapp ^
-DinteractiveMode=false
```

Step 3: Wait for Project Creation
Maven downloads required dependencies.

Step 4: Open Project Folder

Step 5: Verify Project Structure

Step 6: Edit index.jsp
Open:
src\main\webapp\index.js

```
<!DOCTYPE html>
<html>
<head>
   <title>Simple Maven Web App</title>
</head>
```

```
<body bgcolor="lightblue">
   <center>
      <h1>Welcome to Maven Web Application</h1>
      <h2>Hello World!</h2>
      <p> This web application was created using Maven WebApp Archetype
         and deployed on Apache Tomcat Server.
      </p>
      <h3>Features</h3>
      <br>
      Java Web Application<br>  Maven Build Tool<br>
      Apache Tomcat Deployment<br> Simple and Easy to Use<br>
      <br>
      <button>Explore More</button>
      <hr>
   </center>
</body>
</html>
```

Step 7: Build the Application
Execute: **mvn clean package**

Step 8: Verify WAR File
Go to:
SimpleWebApp\target
You will find:
SimpleWebApp.war

Step 9: Deploy WAR File to Tomcat
Copy:
SimpleWebApp.war
Paste into:
apache-tomcat-10.1.57-windows-x64\apache-tomcat-10.1.57\webapps

Step 10: Start Tomcat
Open bin folder:
Double-click: **startup.bat**

Step 11: Verify Tomcat
Open Browser:
If Tomcat uses port 9090:

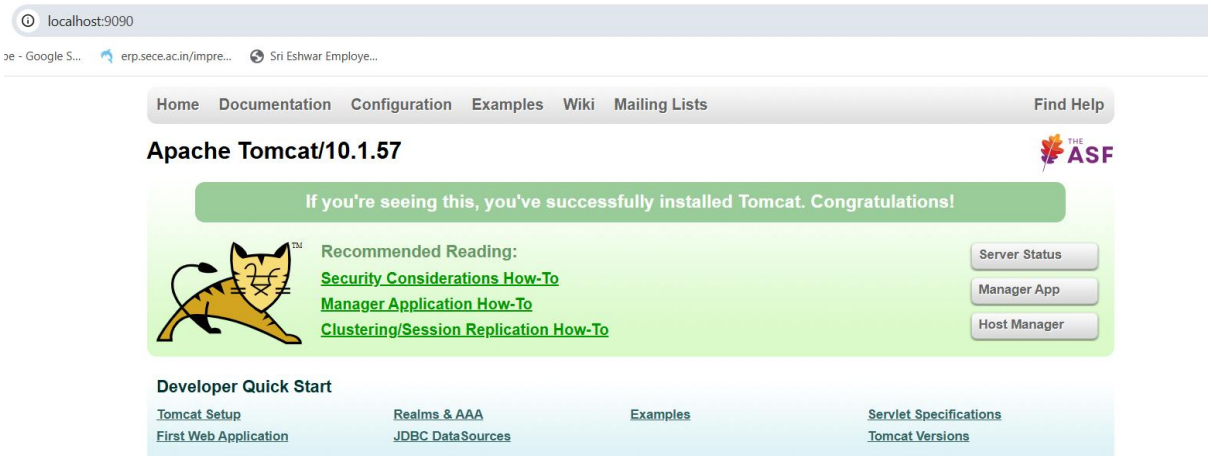
Tomcat Welcome Page should appear.

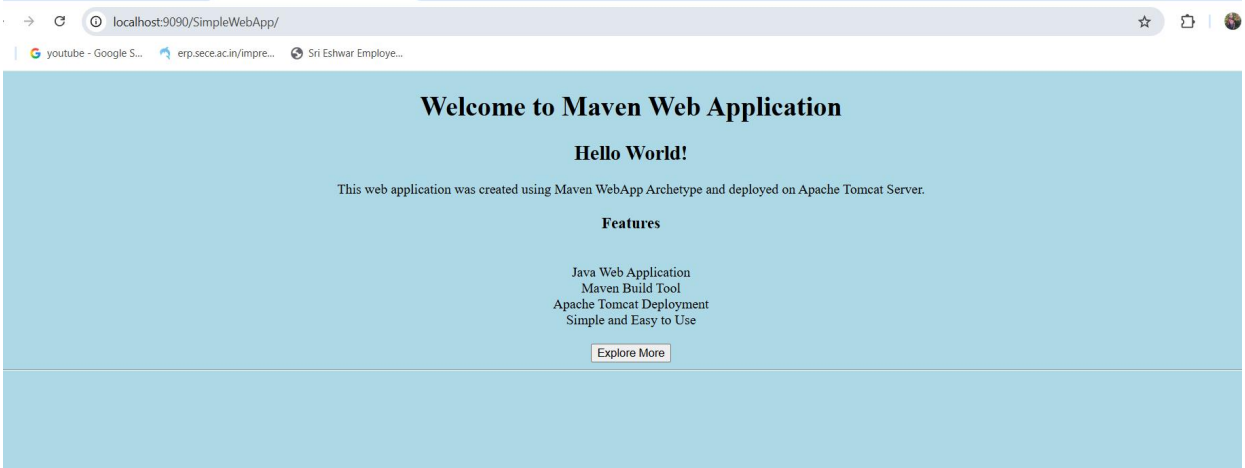
Step 12: Run the Web Application
Open Browser:
http://localhost:9090/SimpleWebApp

Output

```
C:\Users\91904\Desktop>mvn archetype:generate ^
More?  -DgroupId=com.example ^
More? -DartifactId=SimpleWebApp ^
More? -DarchetypeArtifactId=maven-archetype-webapp ^
More? -DarchetypeArtifactId=maven-archetype-webapp ^
More? -DinteractiveMode=false
[INFO] Scanning for projects...
Downloading from central: https://repo.maven.apache.org/maven2/org/apache/maven/plugins/maven-clean-plugin/3.
n-clean-plugin-3.2.0.pom
Downloaded from central: https://repo.maven.apache.org/maven2/org/apache/maven/plugins/maven-clean-plugin/3.2
-clean-plugin-3.2.0.pom (5.3 kB at 4.4 kB/s)
Downloading from central: https://repo.maven.apache.org/maven2/org/apache/maven/plugins/maven-plugins/35/mave
s-35.pom
Downloaded from central: https://repo.maven.apache.org/maven2/org/apache/maven/plugins/maven-plugins/35/maven
-35.pom (9.9 kB at 58 kB/s)
Downloading from central: https://repo.maven.apache.org/maven2/org/apache/maven/maven-parent/35/maven-parent-
Downloaded from central: https://repo.maven.apache.org/maven2/org/apache/maven/maven-parent/35/maven-parent-3
```

```
maven-archetype-webapp-1.0.jar (3.9 kB at 16 kB/s)
[INFO] ----------------------------------------------------------------------------
[INFO] Using following parameters for creating project from Old (1.x) Archetype: maven-archetype-webapp:1.0
[INFO] ----------------------------------------------------------------------------
[INFO] Parameter: basedir, Value: C:\Users\91904\Desktop
[INFO] Parameter: package, Value: com.example
[INFO] Parameter: groupId, Value: com.example
[INFO] Parameter: artifactId, Value: SimpleWebApp
[INFO] Parameter: packageName, Value: com.example
[INFO] Parameter: version, Value: 1.0-SNAPSHOT
[INFO] project created from Old (1.x) Archetype in dir: C:\Users\91904\Desktop\SimpleWebApp
[INFO] ------------------------------------------------------------------------
[INFO] BUILD SUCCESS
[INFO] ------------------------------------------------------------------------
[INFO] Total time:  48.214 s
[INFO] Finished at: 2026-08-02T10:03:31+05:30
[INFO] ------------------------------------------------------------------------
```





SRI ESHWAR COLLEGE OF ENGINEERING DEPARTMENT OF CSE		
PARAMETERS	**MAX MARKS**	**MARKS OBTAINED**
Pre Lab	20	
Program	25	
Output	20	
Viva	10	
Total	75	
SIGNATURE OF THE FACULTY		

Result

A simple Java Web Application was successfully created using the Maven WebApp Archetype. The application was built using Maven, packaged as a WAR file, deployed on the Apache Tomcat Server, and executed successfully through a web browser.

Ex.No: 05	Jenkins pipeline to compile and test a Java project using Maven.
Date:	

Aim

To create and execute a Jenkins Pipeline that compiles and tests a Java project using Maven.

Prerequisites

- Java JDK 21
- Apache Maven 3.9.16
- Jenkins

Procedure

Step 1: Create a Maven Java project

- Navigate to the folder where you want to create the project:

  cd "D:\Java Project"

- Create the Maven project structure manually:

```
mkdir DemoProject
cd DemoProject
mkdir src
mkdir src\main
mkdir src\main\java
mkdir src\test
mkdir src\test\java
```

- Create a pom.xml file in the DemoProject folder.
- Add the required Maven configuration and dependencies in pom.xml.

pom.xml:

```
<project xmlns="http://maven.apache.org/POM/4.0.0"
        xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
        xsi:schemaLocation="http://maven.apache.org/POM/4.0.0
        https://maven.apache.org/xsd/maven-4.0.0.xsd">
    <modelVersion>4.0.0</modelVersion>

    <groupId>com.example</groupId>
    <artifactId>DemoProject</artifactId>
    <version>1.0</version>
    <properties>
        <maven.compiler.source>21</maven.compiler.source>
        <maven.compiler.target>21</maven.compiler.target>
    </properties>

    <dependencies>
        <dependency>
            <groupId>org.junit.jupiter</groupId>
```

```
        <artifactId>junit-jupiter</artifactId>
        <version>5.10.2</version>
        <scope>test</scope>
      </dependency>
    </dependencies>

    <build>
      <plugins>
        <plugin>
          <groupId>org.apache.maven.plugins</groupId>
          <artifactId>maven-surefire-plugin</artifactId>
          <version>3.2.5</version>
        </plugin>
      </plugins>
    </build>
</project>
```

- **Create the Java source file:**

src/main/java/App.java

```
public class App {
    public static void main(String[] args) {

        System.out.println("   Maven Jenkins Pipeline Demo");
        int num1 = 20;
        int num2 = 10;

        System.out.println("Addition: " + add(num1, num2));
        System.out.println("Subtraction: " + subtract(num1, num2));
        System.out.println("Multiplication: " + multiply(num1, num2));
        System.out.println("Division: " + divide(num1, num2));
        System.out.println();
        System.out.println("Student Details");
        System.out.println("------------------------");

        String name = "Harshitha";
        String course = "M.Tech";
        int semester = 4;

        System.out.println("Name      : " + name);
        System.out.println("Course    : " + course);
        System.out.println("Semester  : " + semester);
        System.out.println();
        System.out.println("Program Executed Successfully");
    }
    public static int add(int a, int b) {
        return a + b;
    }

    public static int subtract(int a, int b) {
        return a - b;
    }

    public static int multiply(int a, int b) {
        return a * b;
    }
```

```
    public static double divide(int a, int b) {
        return (double) a / b;
    }
}
```

➢ **Create the test file:**

src/test/java/AppTest.java

```
import org.junit.jupiter.api.Test;
import static org.junit.jupiter.api.Assertions.*;

public class AppTest {
    @Test
    public void testAddition() {
        assertEquals(30, App.add(20, 10));
    }

    @Test
    public void testSubtraction() {
        assertEquals(10, App.subtract(20, 10));
    }

    @Test
    public void testMultiplication() {
        assertEquals(200, App.multiply(20, 10));
    }

    @Test
    public void testDivision() {
        assertEquals(2.0, App.divide(20, 10));
    }
}
```

Step 2: Validate and test the Maven Project

Command: **mvn validate**
mvn clean test

Step 3: Create a Jenkins Pipeline Job

- Open Jenkins in a web browser. http://localhost:8080/
- Click New Item.
- Enter the job name: **JavaPipeline**
- Select Pipeline and click OK.

Step 4: Configure Pipeline Script

- In the Pipeline section, choose Pipeline Script.
- **Enter the Pipeline script.**

```
pipeline {
    agent any

    stages {
```

```
        stage('Compile') {
            steps {
                dir('D:/Java Project/DemoProject') {
                    bat '"C:\\Users\\91904\\Downloads\\apache-maven-3.9.16-bin\\apache-maven-3.9.16\\bin\\mvn.cmd" compile'
                }
            }
        }

        stage('Test') {
            steps {
                dir('D:/Java Project/DemoProject') {
                    bat '"C:\\Users\\91904\\Downloads\\apache-maven-3.9.16-bin\\apache-maven-3.9.16\\bin\\mvn.cmd" test'
                }
            }
        }
    }
}
```

- Click **Save**.

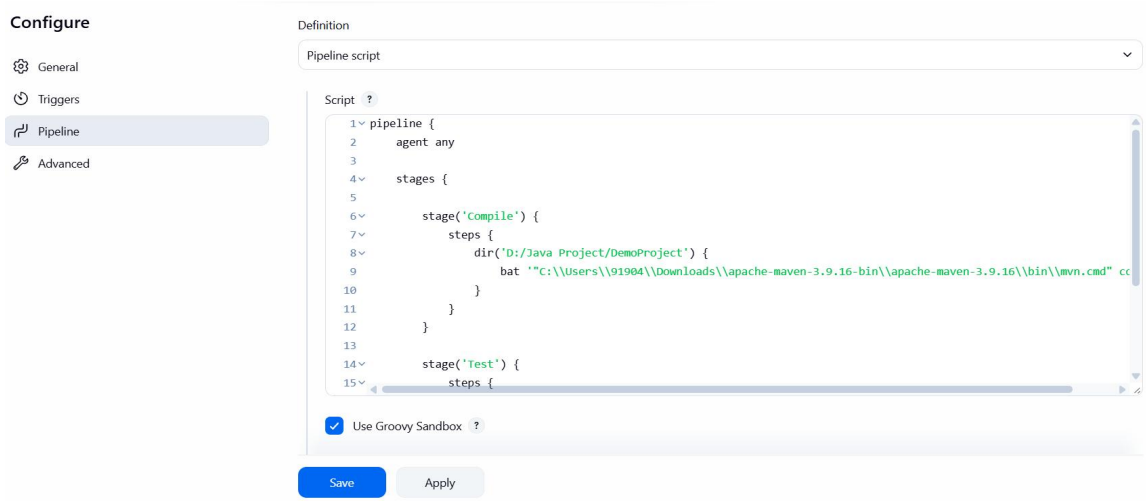
Step 5: Click Build Now

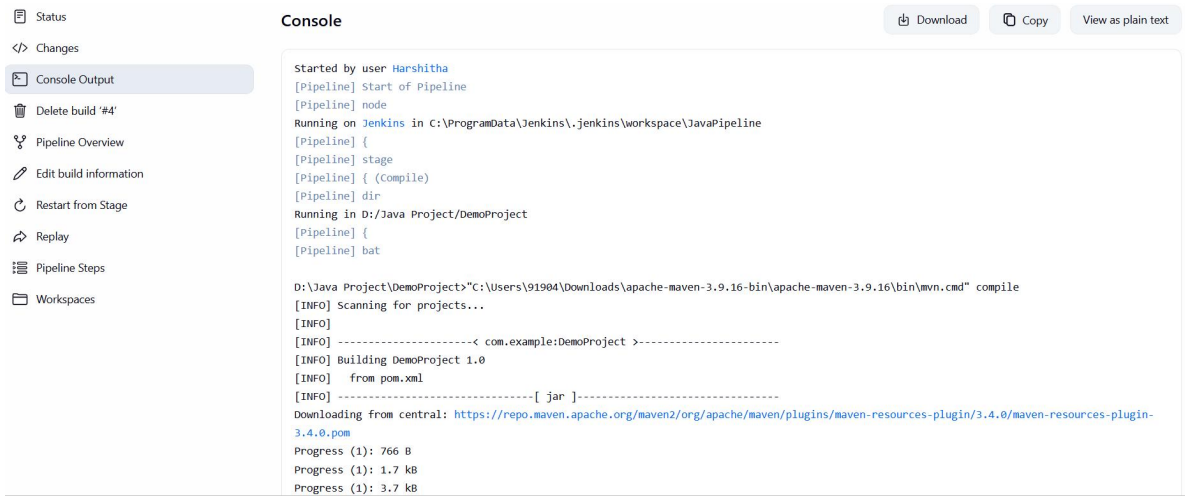
Step 6: View Console Output and verify successful compilation and test execution.

Output

```
PS D:\Java Project\DemoProject> mvn validate
[INFO] Scanning for projects...
[INFO]
[INFO] ----------------------< com.example:DemoProject >-----------------------
[INFO] Building DemoProject 1.0
[INFO]   from pom.xml
[INFO] --------------------------------[ jar ]---------------------------------
[INFO] ------------------------------------------------------------------------
[INFO] BUILD SUCCESS
[INFO] ------------------------------------------------------------------------
[INFO] Total time:  0.202 s
[INFO] Finished at: 2026-08-02T14:12:57+05:30
[INFO] ------------------------------------------------------------------------
```

```
PS D:\Java Project\DemoProject> mvn clean test
[INFO] Scanning for projects...
[INFO]
[INFO] ----------------------< com.example:DemoProject >-----------------------
[INFO] Building DemoProject 1.0
[INFO]   from pom.xml
[INFO] --------------------------------[ jar ]---------------------------------
[INFO] -------------------------------------------------------
[INFO]  T E S T S
[INFO] -------------------------------------------------------
[INFO] Running AppTest
[INFO] Tests run: 4, Failures: 0, Errors: 0, Skipped: 0, Time elapsed: 0.083 s -- in AppTest
[INFO]
[INFO] Results:
[INFO]
[INFO] Tests run: 4, Failures: 0, Errors: 0, Skipped: 0
[INFO]
[INFO] ------------------------------------------------------------------------
[INFO] BUILD SUCCESS
[INFO] ------------------------------------------------------------------------
[INFO] Total time:  29.116 s
[INFO] Finished at: 2026-08-02T14:24:34+05:30
[INFO] ------------------------------------------------------------------------
```






```
[INFO] -------------------------------------------------------
[INFO]  T E S T S
[INFO] -------------------------------------------------------
[INFO] Running AppTest
[INFO] Tests run: 4, Failures: 0, Errors: 0, Skipped: 0, Time elapsed: 0.084 s -- in AppTest
[INFO]
[INFO] Results:
[INFO]
[INFO] Tests run: 4, Failures: 0, Errors: 0, Skipped: 0
[INFO]
[INFO] ------------------------------------------------------------------------
[INFO] BUILD SUCCESS
[INFO] ------------------------------------------------------------------------
[INFO] Total time:  31.611 s
[INFO] Finished at: 2026-08-02T14:48:32+05:30
[INFO] ------------------------------------------------------------------------
[Pipeline] }
[Pipeline] // dir
[Pipeline] }
[Pipeline] // stage
[Pipeline] }
[Pipeline] // node
[Pipeline] End of Pipeline
Finished: SUCCESS
```

SRI ESHWAR COLLEGE OF ENGINEERING DEPARTMENT OF CSE		
PARAMETERS	**MAX MARKS**	**MARKS OBTAINED**
Pre Lab	20	
Program	25	
Output	20	
Viva	10	
Total	75	
SIGNATURE OF THE FACULTY		

Result

The Jenkins Pipeline was successfully created and executed. The Java Maven project was compiled and tested successfully using Maven commands within Jenkins. All test cases passed, and the build completed with BUILD SUCCESS status.

Ex.No:06	Jenkins credentials to securely access private GitHub repositories during CI builds
Date:	

Aim

To configure Jenkins credentials for secure access to a private GitHub repository during Continuous Integration (CI) builds.

Prerequisites

- Jenkins
- Git
- GitHub Account
- Personal Access Token (PAT)

Procedure

Step 1: Generate a GitHub Personal Access Token (PAT)

- Log in to GitHub.
- Go to Settings → Developer Settings → Personal Access Tokens → Tokens (Classic).
- Click Generate New Token.
- Generate and copy the token.

Step 2: Open Jenkins Credentials Manager

- Open Jenkins Dashboard.
- Navigate to:

Manage Jenkins → Credentials

Step 3: Add GitHub Credentials

- Select **System → Global Credentials (unrestricted)**.
- Click **Add Credentials**.
- Configure:
  - **Kind:** Username with Password
  - **Username:** GitHub Username
  - **Password:** GitHub Personal Access Token (PAT)
  - **ID:** github-credentials
  - **Description:** GitHub Access Token
- Click **Create.**

Step 4: Create or Configure Jenkins Job

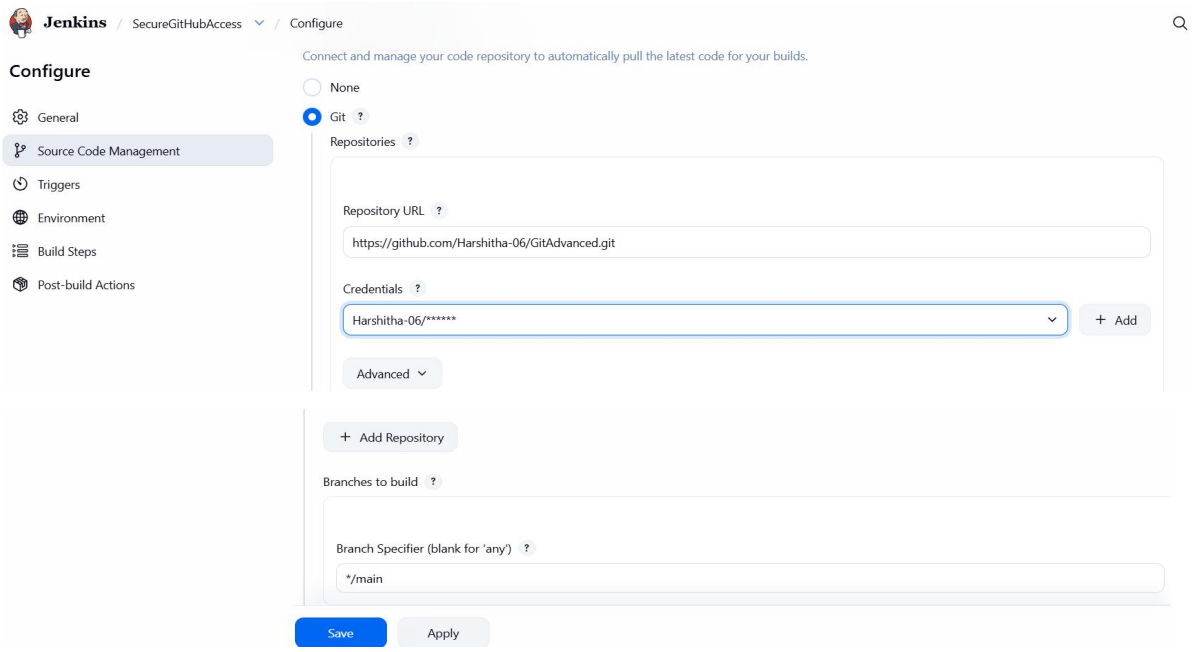
1. Create a new Jenkins job or open an existing job.
2. In **Source Code Management**, select **Git**.
3. Enter the private repository URL.

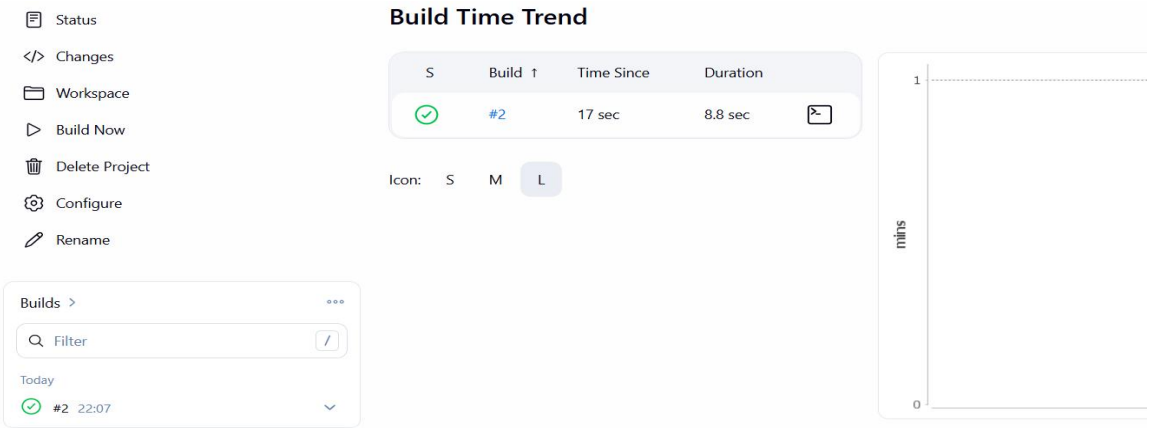
4. Under **Credentials**, select github-credentials.

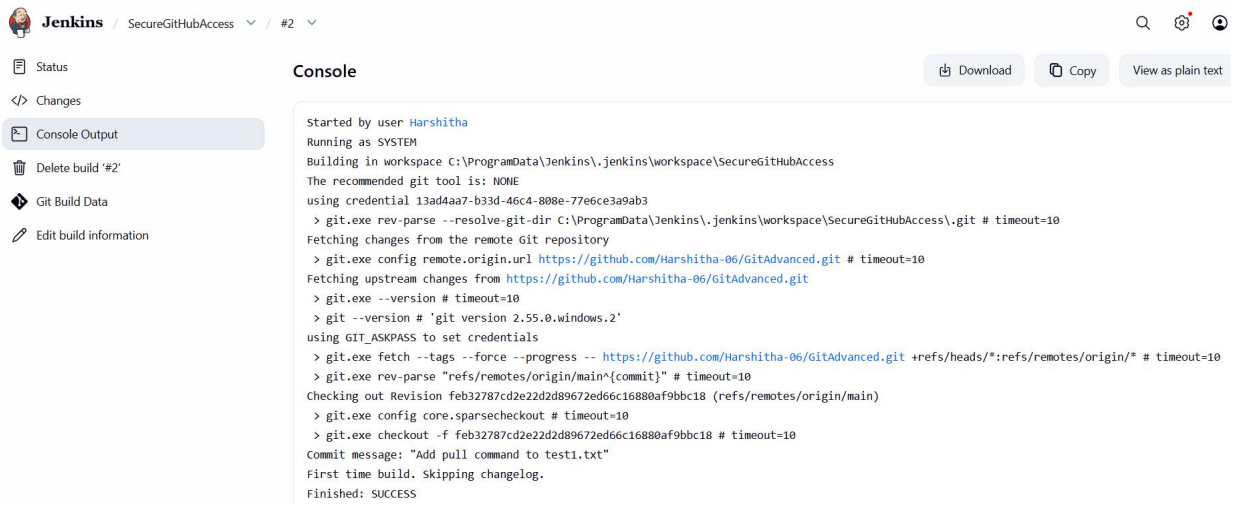
Step 5: Save and Build the Job

1. Click **Save**.
2. Click **Build Now**.
3. Verify that Jenkins successfully clones the private repository.

Output







SRI ESHWAR COLLEGE OF ENGINEERING DEPARTMENT OF CSE		
PARAMETERS	**MAX MARKS**	**MARKS OBTAINED**
Pre Lab	20	
Program	25	
Output	20	
Viva	10	
Total	75	
SIGNATURE OF THE FACULTY		

Result

Jenkins credentials were successfully configured to securely access a private GitHub repository, enabling authenticated repository cloning during CI builds.

Ex.No: 07	Creation of AWS EC2 Instance with Security Group Using Terraform
Date:	

Aim

To provision an AWS EC2 instance with a Security Group and Elastic IP using Terraform.

Prerequisites

- AWS Account
- Terraform Installed
- AWS Access Key and Secret Key
- AWS CLI Configured

Procedure

Step 1: Install Terraform on the local machine and verify the installation using:
https://developer.hashicorp.com/terraform/install

- Select **Windows (amd64)**.

➢ Download the ZIP file.

➢ Extract the ZIP

➢ Add Terraform to Path

➢ Verify Installation

Step 2: Create a project folder and create a Terraform configuration file (main.tf).

In the same folder, create a file named:
main.tf
Open it with Notepad:

Paste:

```
provider "aws" {
  region = "ap-south-1"
}
resource "aws_security_group" "web_sg" {
  name        = "web-security-group"
  description = "Allow SSH Access"

  ingress {
    from_port   = 22
    to_port     = 22
    protocol    = "tcp"
    cidr_blocks = ["0.0.0.0/0"]
  }

  egress {
    from_port   = 0
```

```
    to_port     = 0

    protocol    = "-1"
    cidr_blocks = ["0.0.0.0/0"]
  }
}
resource "aws_instance" "web_server" {
  ami           = "ami-0f58b397bc5c1f2e8"
  instance_type = "t3.micro"

  vpc_security_group_ids = [
    aws_security_group.web_sg.id
  ]
  tags = {
    Name = "Terraform-EC2"
  }
}
resource "aws_eip" "elastic_ip" {
  instance = aws_instance.web_server.id

  tags = {
    Name = "Terraform-EIP"
  }
}
```

Save and close the file.

Step 3: Run Terraform:

Command: **terraform init**

Step 4: Configure AWS credentials using AWS CLI:

i. To use aws configure, you first need to have an **AWS account** and create **Access Keys**.

- Go to **https://aws.amazon.com/**
- Create an **AWS account**
- Complete your registration: 5 Steps

ii. Install AWS CLI

- Download AWS CLI from: https://docs.aws.amazon.com/cli/latest/userguide/getting-started-install.html?
- For Windows, the easiest method is to open **PowerShell as Administrator** and run: **irm https://awscli.amazonaws.com/v2/install.ps1 | iex**

iii. Verify Installation:

- Open a **new Command Prompt** and run:
  aws –version

iv. To Create Access Key:

- Sign in to the AWS Management Console.
- Open the IAM service.
- Create or select an IAM user.

- Go to **Security credentials**.
- Click **Create access key**.
- AWS will provide:
    - Access Key ID
    - Secret Access Key
- **Then run:**
  aws configure
  You can run aws configure in either Command Prompt (CMD) or PowerShell.
- **Verify**
  Run:
  aws sts get-caller-identity
  If successful, AWS will return your Account ID and user information.

Step 5: Initialize the Terraform working directory

Command: **terraform init**

Step 6: Validate the Terraform configuration

Command: **terraform validate**

Step 7: Preview the resources to be created

Command: **terraform plan**

Step 8: Apply the configuration to provision the resources

Command: **terraform apply**

Step 9: Verify in AWS Console
Go to the AWS EC2 Console
Check:

- EC2 → Instances → Terraform-EC2

  Region should be : ap-south-1
- EC2 → Security Groups → web-security-group
- EC2 → Elastic IPs → Terraform-EIP

Output

```
C:\Users\91904>aws configure

Tip: You can deliver temporary credentials to the AWS CLI using your AWS C
in'.

AWS Access Key ID [None]: AKIAXHXSL4HJIFZMGF4P
AWS Secret Access Key [None]: cs47GQPZFX4yliB+KH1QL4yvxDwonNr4xk8vt1Fa
Default region name [None]: ap-south-1
Default output format [None]: json
```


```
C:\Users\91904>aws configure

Tip: You can deliver temporary credentials to the AWS CLI using your AWS C
in'.

AWS Access Key ID [None]: AKIAXHXSL4HJIFZMGF4P
AWS Secret Access Key [None]: cs47GQPZFX4yliB+KH1QL4yvxDwonNr4xk8vt1Fa
Default region name [None]: ap-south-1
Default output format [None]: json
```


```
D:\Terraform\Terraform_Experiment>terraform validate
Success! The configuration is valid.
```

```
D:\Terraform\Terraform_Experiment>terraform plan

Terraform used the selected providers to generate the following execution plan.
following symbols:
  + create

Terraform will perform the following actions:

  # aws_eip.elastic_ip will be created
  + resource "aws_eip" "elastic_ip" {
      + allocation_id        = (known after apply)
      + arn                  = (known after apply)
      + association_id       = (known after apply)
      + carrier_ip           = (known after apply)
      + customer_owned_ip    = (known after apply)
      + domain               = (known after apply)
      + id                   = (known after apply)
      + instance             = (known after apply)
```

```
        ]
      + name                   = "web-security-group"
      + name_prefix            = (known after apply)
      + owner_id               = (known after apply)
      + region                 = "ap-south-1"
      + revoke_rules_on_delete = false
      + tags_all               = (known after apply)
      + vpc_id                 = (known after apply)
    }

Plan: 3 to add, 0 to change, 0 to destroy.

─────────────────────────────────────────────────────────────────────────────

Note: You didn't use the -out option to save this plan, so Terraform can't guarantee
run "terraform apply" now.
```

```
D:\Terraform\Terraform_Experiment>terraform apply

Terraform used the selected providers to generate the following execution plan. Resource ac
following symbols:
  + create

Terraform will perform the following actions:

  # aws_eip.elastic_ip will be created
  + resource "aws_eip" "elastic_ip" {
      + allocation_id        = (known after apply)
      + arn                  = (known after apply)
      + association_id       = (known after apply)
      + carrier_ip           = (known after apply)
      + customer_owned_ip    = (known after apply)
      + domain               = (known after apply)
      + id                   = (known after apply)
      + instance             = (known after apply)
```

```
      + vpc_id                 = (known after apply)
    }

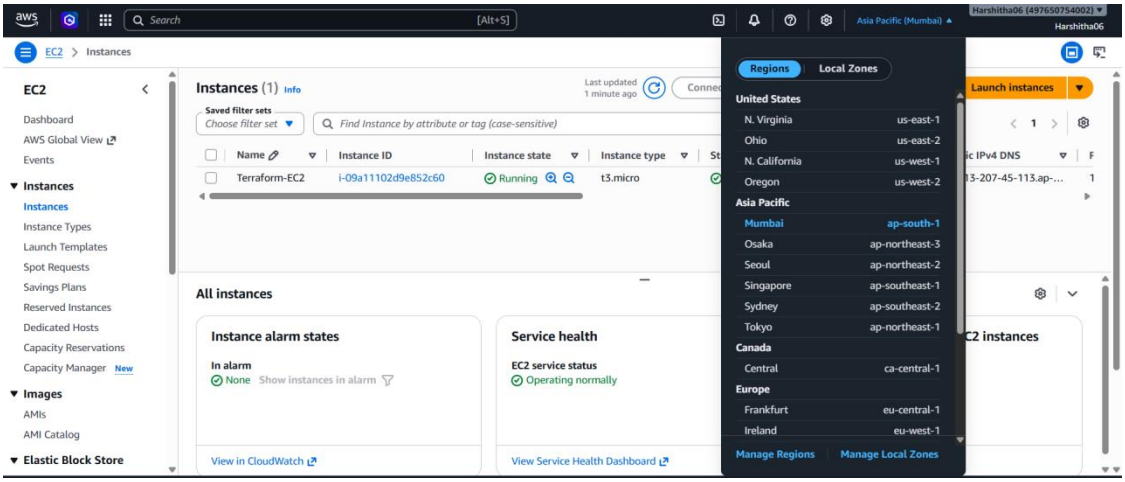
Plan: 3 to add, 0 to change, 0 to destroy.

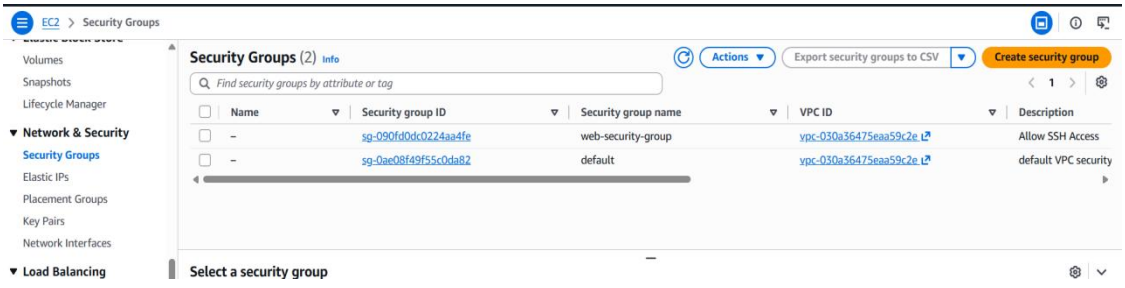
Do you want to perform these actions?
  Terraform will perform the actions described above.
  Only 'yes' will be accepted to approve.

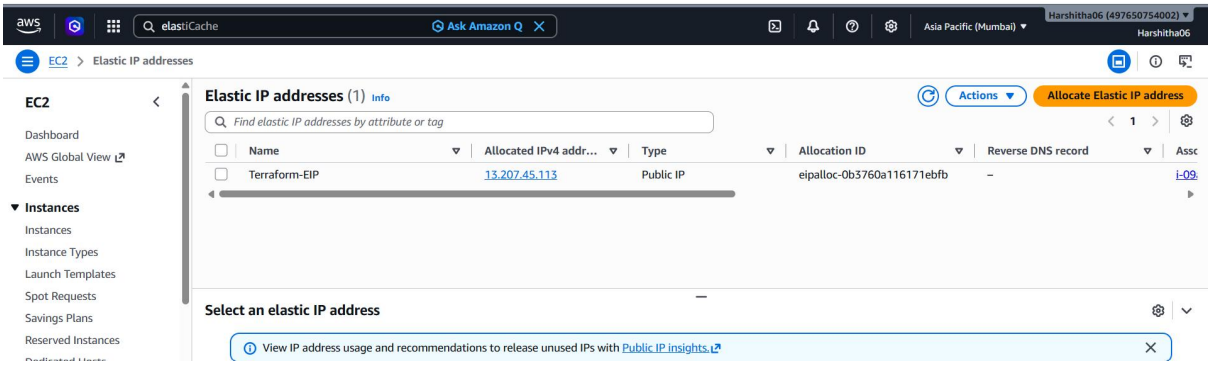
  Enter a value: yes

aws_security_group.web_sg: Creating...
aws_security_group.web_sg: Creation complete after 3s [id=sg-090fd0dc0224aa4fe]
aws_instance.web_server: Creating...
aws_instance.web_server: Still creating... [00m10s elapsed]
aws_instance.web_server: Creation complete after 13s [id=i-09a11102d9e852c60]
aws_eip.elastic_ip: Creating...
aws_eip.elastic_ip: Creation complete after 2s [id=eipalloc-0b3760a116171ebfb]

Apply complete! Resources: 3 added, 0 changed, 0 destroyed.
```







SRI ESHWAR COLLEGE OF ENGINEERING DEPARTMENT OF CSE		
PARAMETERS	**MAX MARKS**	**MARKS OBTAINED**
Pre Lab	20	
Program	25	
Output	20	
Viva	10	
Total	75	
SIGNATURE OF THE FACULTY		

Result

An AWS EC2 instance was successfully provisioned using Terraform. A Security Group allowing SSH access was attached to the instance, and an Elastic IP address was allocated and associated with the EC2 instance.

Ex.No: 08	Create a playbook to manage users and groups with custom SSH key distribution.
Date:	

Aim

To create and execute an Ansible playbook that automates the creation of users and groups and distributes custom SSH public keys for secure passwordless authentication.

Prerequisites

- WSL (Windows Subsystem for Linux) with Ubuntu installed
- Ansible installed on Ubuntu
- SSH configured
- Inventory file

Procedure

Step 1: Install WSL

Open PowerShell as Administrator:

wsl -- install

Restart the system.

Step 2: Open Ubuntu (WSL)

Open Ubuntu from the Start Menu and create a username and password.

Step 3: Install Ansible

Update Packages:

sudo apt update

sudo apt install ansible -y

Verify Installation:

ansible --version

Step 4: Create Project Directory

Step 5: Create Inventory File

nano inventory.ini

Paste inside the file:

[local]

localhost ansible_connection=local

Step 6: Create the Playbook

nano user_group_ssh.yml

Paste the code inside the file:

```
---
- hosts: local

  tasks:

    - name: Create group
      group:
```

```
    name: developers
    state: present

- name: Create user
  user:
    name: devuser
    group: developers
    create_home: yes
    state: present

- name: Create .ssh directory
  file:
    path: /home/devuser/.ssh
    state: directory
    owner: devuser
    group: developers
    mode: '0700'
- name: Add SSH public key
  authorized_key:
    user: devuser
    state: present
    key: "ssh-rsa AAAAB3NzaC1yc2EAAAADAQABAAABAQCExampleSSHKey"
```

Verify Files:
ls

Generate SSH Key:

Command: **ssh -keygen**

Paste the Key inside the playbook file.

Next Step: View the Public Key
cat ~/.ssh/id_ed25519.pub

Step 7: Run the Playbook
sudo ansible-playbook -i inventory.ini user_group_ssh.yml

Step 8: Verify User Creation
After success:
id devuser

Step 9: Verify SSH Key Distribution

Output

```
harshitha@DESKTOP-HL1EO0Q:~$ sudo apt install ansible -y
Installing:
  ansible

Installing dependencies:
  ansible-core  python3-argcomplete  python3-dnspython  python3-jmespath  python3-libcloud
  ieee-data     python3-decorator    python3-gssapi     python3-kerberos  python3-netaddr

Suggested packages:
  cowsay  sshpass  python3-aioquic  python3-h2  python3-httpcore  python3-httpx  python3-tr

Summary:
  Upgrading: 0, Installing: 17, Removing: 0, Not Upgrading: 110
  Download size: 25.1 MB
  Space needed: 262 MB / 1024 GB available

Get:1 http://archive.ubuntu.com/ubuntu resolute/universe amd64 python3-resolvelib all 1.2.1
```

```
harshitha@DESKTOP-HL1EO0Q:~/ansible-lab$ cd /mnt/d
mkdir ansible-lab
cd ansible-lab
harshitha@DESKTOP-HL1EO0Q:/mnt/d/ansible-lab$
```

```
harshitha@DESKTOP-HL1EO0Q:/mnt/d/ansible-1$ ssh-keygen
Generating public/private ed25519 key pair.
Enter file in which to save the key (/home/harshitha/.ssh/id_ed25519):
Created directory '/home/harshitha/.ssh'.
Enter passphrase for "/home/harshitha/.ssh/id_ed25519" (empty for no passphrase):
Enter same passphrase again:
Your identification has been saved in /home/harshitha/.ssh/id_ed25519
Your public key has been saved in /home/harshitha/.ssh/id_ed25519.pub
The key fingerprint is:
SHA256:nnmd+g96OFazWSP7HfPUz28gZQ8jk/B4/Qx9iCy4j8c harshitha@DESKTOP-HL1EO0Q
The key's randomart image is:
+--[ED25519 256]--+
|                 |
|           .     |
|          . = + o |
|         . o O O o|
|        S. o = B.|
|       ..o .=.+ =|
|        ++.o+O =o|
|        ..Eo=. +*|
|         oo+.oo.B|
+----[SHA256]-----+
```

```
harshitha@DESKTOP-HL1EO0Q:/mnt/d/ansible-lab$ sudo ansible-playbook -i inventory.ini user_group_ssh.yml

PLAY [local] ***********************************************************************************************************

TASK [Gathering Facts] *************************************************************************************************
[WARNING]: Host 'localhost' is using the discovered Python interpreter at '/usr/bin/python3.14', but future install
 cause a different interpreter to be discovered. See https://docs.ansible.com/ansible-core/2.20/reference_appendice
ormation.
ok: [localhost]

TASK [Create group] ****************************************************************************************************
ok: [localhost]

TASK [Create user] *****************************************************************************************************
ok: [localhost]

TASK [Create .ssh directory] *******************************************************************************************
changed: [localhost]

TASK [Add SSH public key] **********************************************************************************************
[WARNING]: Deprecation warnings can be disabled by setting `deprecation_warnings=False` in ansible.cfg.
[DEPRECATION WARNING]: Importing 'to_native' from 'ansible.module_utils._text' is deprecated. This feature will be
se ansible.module_utils.common.text.converters instead.
changed: [localhost]

PLAY RECAP *************************************************************************************************************
localhost                  : ok=5    changed=2    unreachable=0    failed=0    skipped=0    rescued=0    ignored=0
```

```
harshitha@DESKTOP-HL1EO0Q:/mnt/d/ansible-lab$ id devuser
uid=1001(devuser) gid=1001(developers) groups=1001(developers)
harshitha@DESKTOP-HL1EO0Q:/mnt/d/ansible-lab$ sudo cat /home/devuser/.ssh/authorized_keys
ssh-rsa AAAAB3NzaC1yc2EAAAADAQABAAABAQCExampleSSHKey
```

SRI ESHWAR COLLEGE OF ENGINEERING DEPARTMENT OF CSE		
PARAMETERS	**MAX MARKS**	**MARKS OBTAINED**
Pre Lab	20	
Program	25	
Output	20	
Viva	10	
Total	75	
SIGNATURE OF THE FACULTY		

Result

Thus, the Ansible playbook was successfully executed to automate user and group management and distribute custom SSH public keys. The user account, group, SSH directory, and authorized key were created successfully, enabling passwordless SSH authentication.

Ex.No: 09	Deploy Grafana and Integrate it with Prometheus as a Data Source to Visualize Metrics
Date:	

Aim

To deploy Grafana and integrate it with Prometheus as a data source for visualizing and monitoring system and application metrics through interactive dashboards.

Prerequisites

- Ubuntu/Linux System
- Prometheus Configured
- Grafana Installed

Procedure

Step 1: Install Prometheus

Download Prometheus for Windows from:
https://prometheus.io/download/
Download:
prometheus-3.13.2.windows-amd64.zip

➢ Extract the zip file.

Step 2: Verify Prometheus is Running

Start Prometheus:

.\prometheus.exe

Open a browser and access:
http://localhost:9090

Step 3: Install Grafana

Download Grafana for Windows from:
Grafana Downloads
Choose:
grafana-enterprise_13.1.3_31135815010_windows_amd64.msi

Installation Steps:

1. Click **Download the installer**.
2. Run the .msi file.
3. Click **Next → Next → Install**.

Step 4: Start Grafana

Open your browser and visit:
http://localhost:3000
If Grafana is running, you'll see the login page.
Login Credentials
Username: admin
Password: admin

Step 5: Add Prometheus as Data Source:

1. Click **Connections → Data Sources**
2. Click **Add Data Source**
3. Select **Prometheus**
4. Enter:

   http://localhost:9090

5. Click **Save & Test**

Step 6: Create Dashboard:

1. Dashboards → New Dashboard
2. Add Visualization

   Click **"Add a panel to visualize your data"**

   Click Configure Visualization

3. Select Prometheus

   If Prometheus is connected correctly, it will appear in the dropdown.

4. In Query **A**:

   - Click **Metric**

     Type:

     up

   - Select **up** from the list.

5. Click **Run queries**.
6. Save Dashboard

Step 7: Add CPU Monitoring Panel:

1. Open your dashboard.
2. Click **Add → Visualization**.
3. Select **Prometheus**.
4. Enter:

   process_cpu_seconds_total

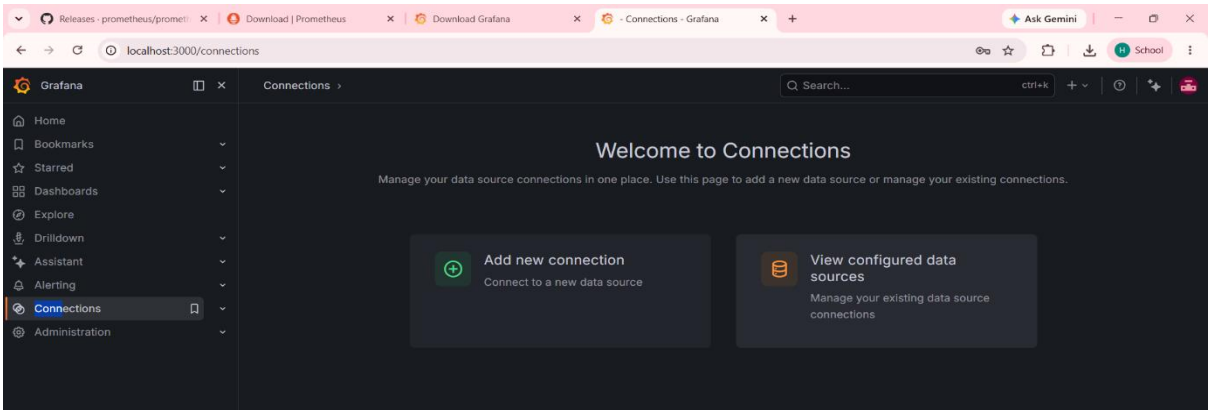
5. Click **Run Query**.
6. Click **Apply**.

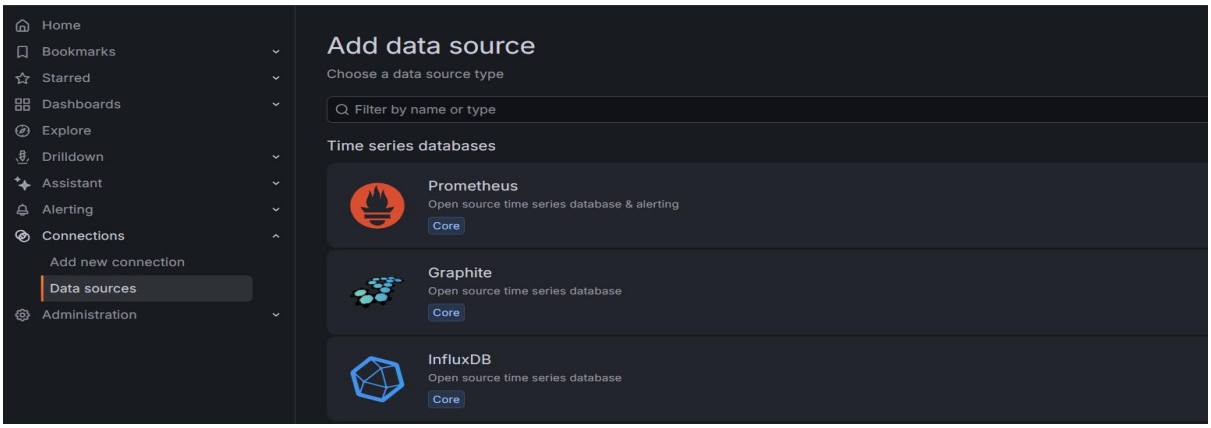
Step 8: Add Memory Monitoring Panel:

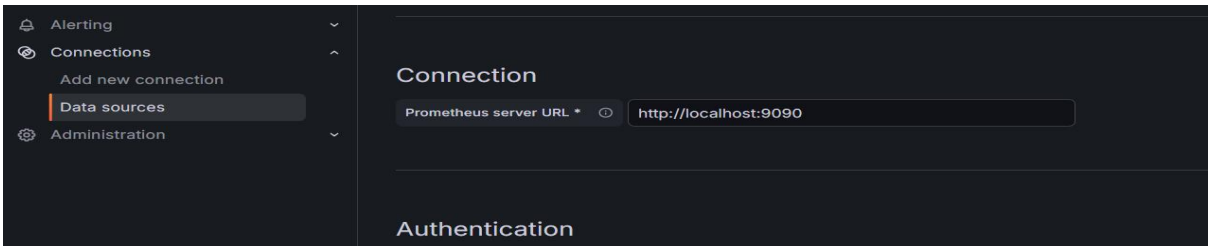
1. Click **Add → Visualization**.
2. Select **Prometheus**.
3. Enter:

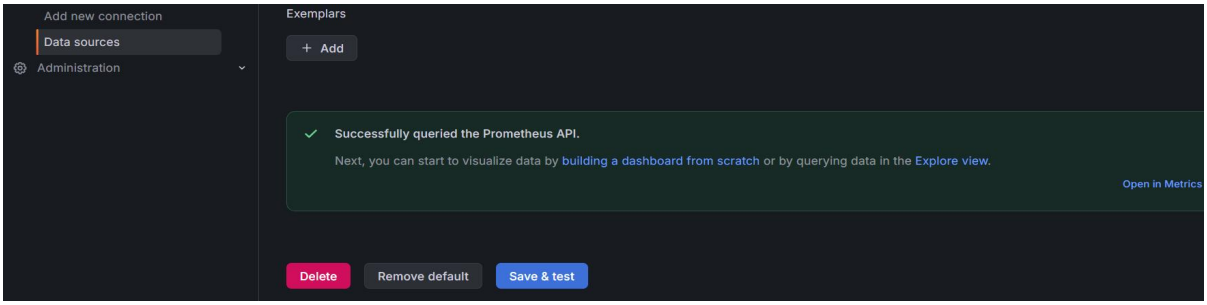
   go_memstats_alloc_bytes
4. Click **Run Query**.
5. Click **Apply**.

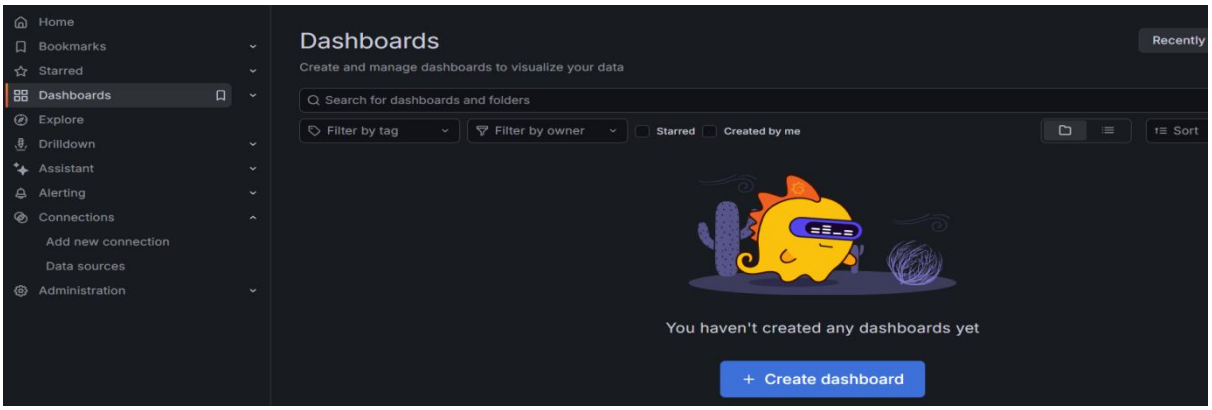
Output

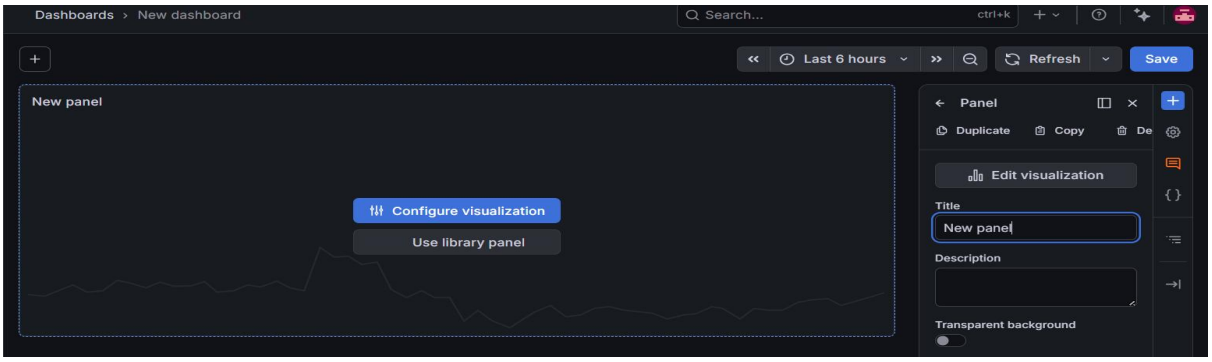


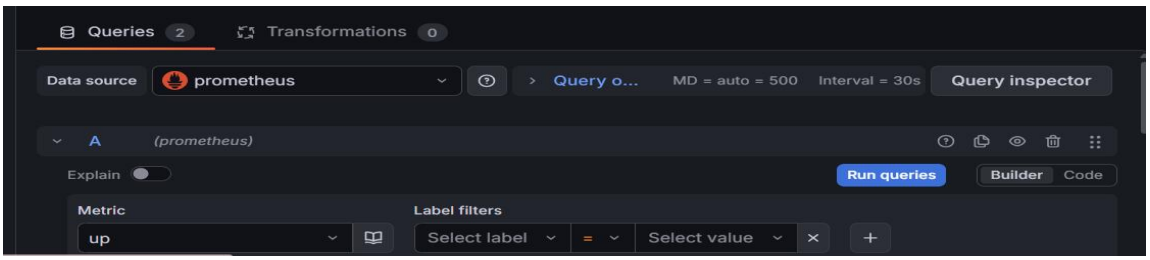


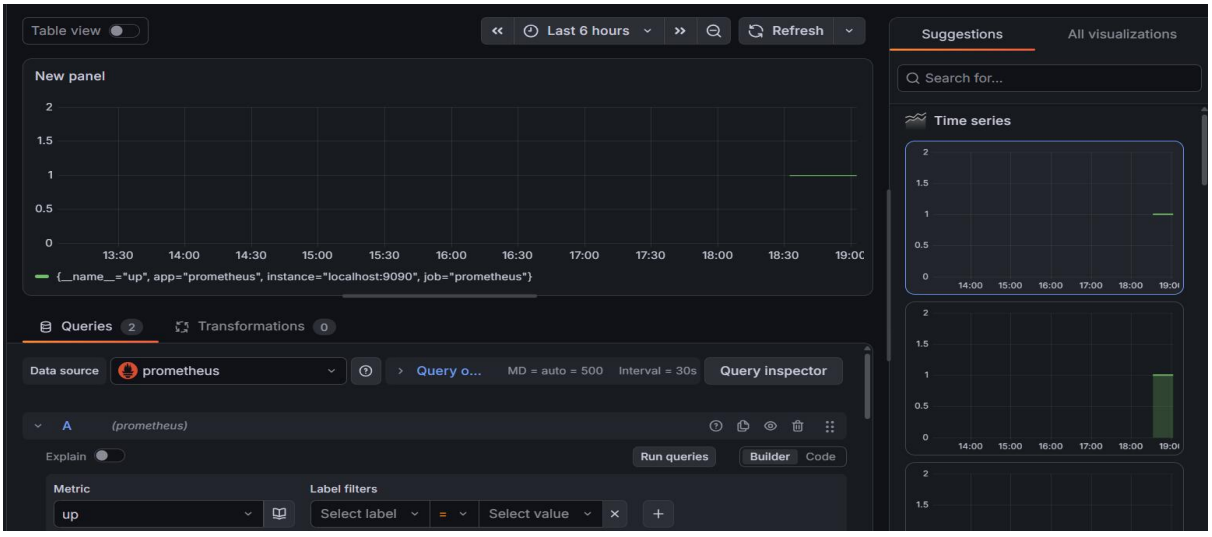


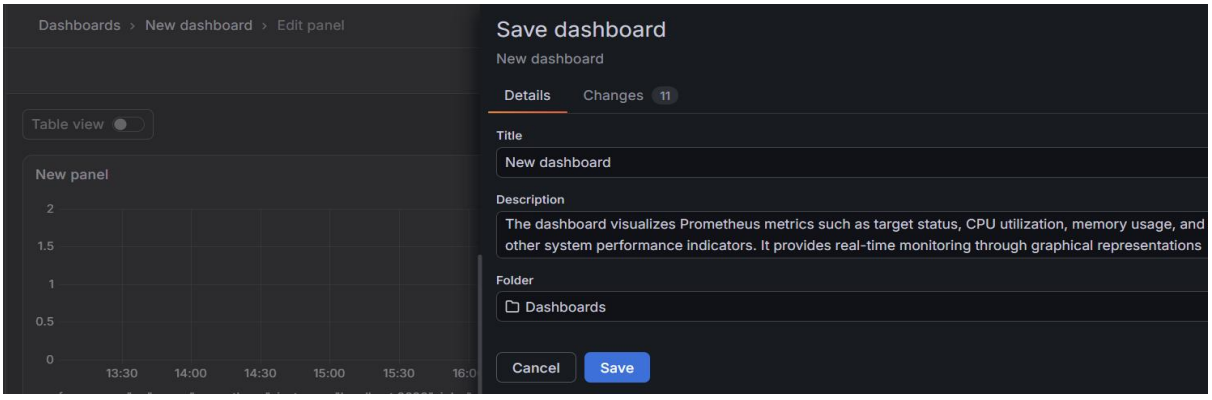


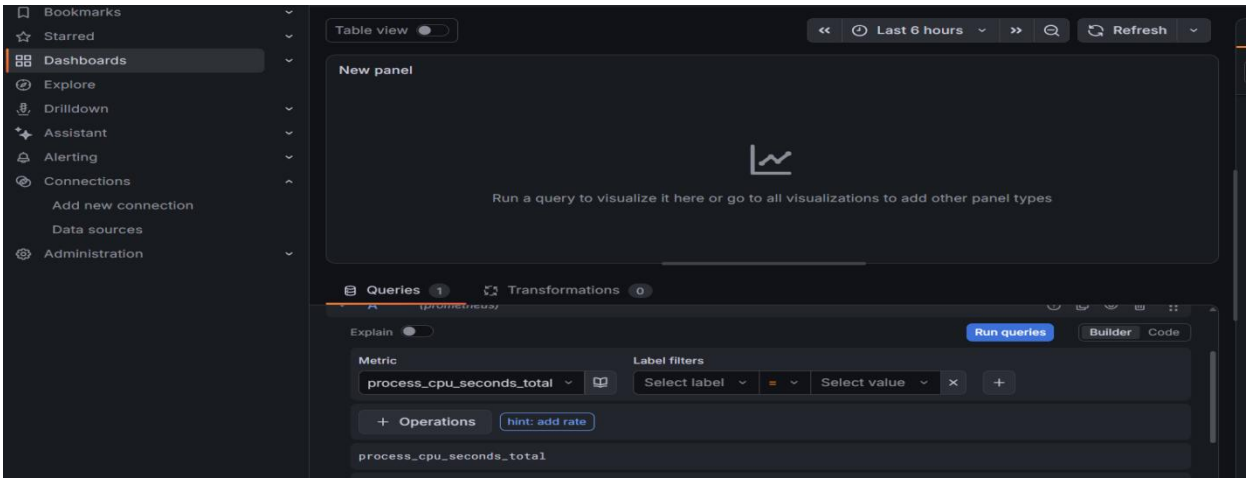

Home
Bookmarks
Starred
Dashboards
Explore
Drilldown
Assistant
Alerting
Connections
Add new connection
Data sources
Administration
Dashboards
Create and manage dashboards to visualize your data
Recently
Search for dashboards and folders
Filter by tag
Filter by owner
Starred
Created by me
Sort
You haven't created any dashboards yet
+ Create dashboard

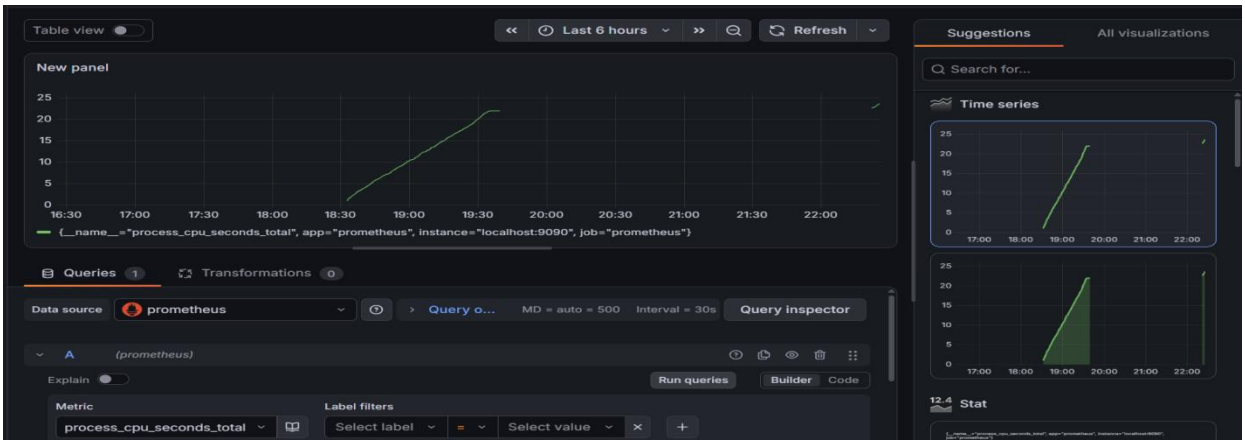

Dashboards › New dashboard
Search...
ctrl+k
Last 6 hours
Refresh
Save
New panel
Configure visualization
Use library panel
Panel
Duplicate
Copy
Edit visualization
Title
New panel
Description
Transparent background

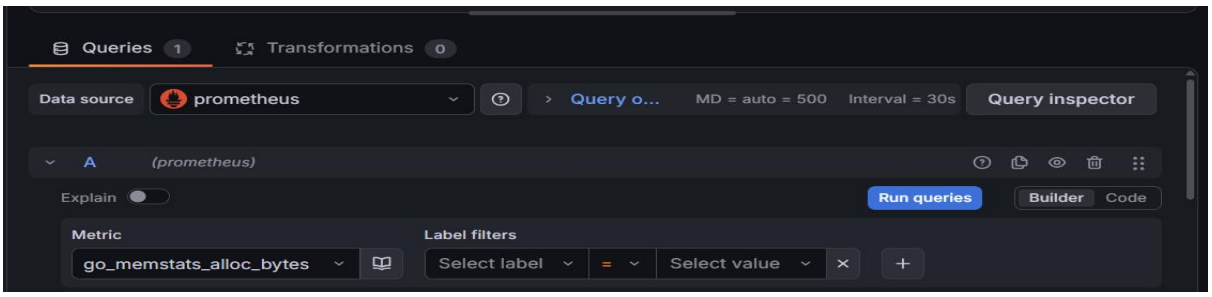

Queries 2
Transformations 0
Data source
prometheus
Query o...
MD = auto = 500
Interval = 30s
Query inspector
A
(prometheus)
Explain
Run queries
Builder
Code
Metric
Label filters
up
Select label
Select value

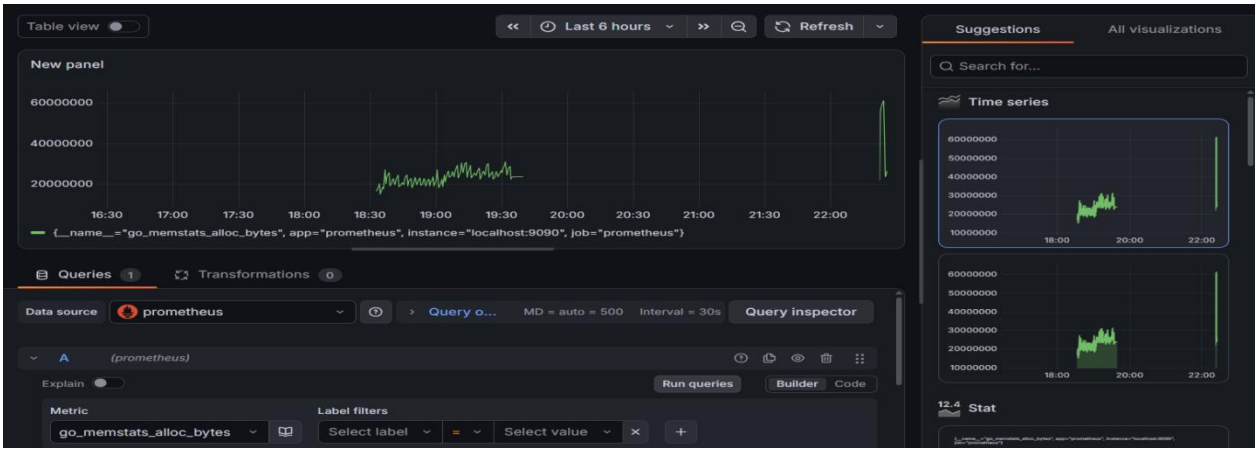

Table view
Last 6 hours
Refresh
New panel
2
1.5
1
0.5
0
13:30
14:00
14:30
15:00
15:30
16:00
16:30
17:00
17:30
18:00
18:30
{__name__="up", app="prometheus", instance="localhost:9090", job="prometheus"}
Queries 2
Transformations 0
Data source
prometheus
Query o...
MD = auto = 500
Interval = 30s
Query inspector
A
(prometheus)
Explain
Run queries
Builder
Code
Metric
Label filters
up
Select label
Select value
Suggestions
All visualizations
Search for...
Time series


Dashboards › New dashboard › Edit panel
Table view
New panel
2
1.5
1
0.5
0
13:30
14:00
14:30
15:00
15:30
Save dashboard
New dashboard
Details
Changes 11
Title
New dashboard
Description
The dashboard visualizes Prometheus metrics such as target status, CPU utilization, memory usage, and other system performance indicators. It provides real-time monitoring through graphical representations
Folder
Dashboards
Cancel
Save


Bookmarks
Starred
Dashboards
Explore
Drilldown
Assistant
Alerting
Connections
Add new connection
Data sources
Administration
Table view
Last 6 hours
Refresh
New panel
Run a query to visualize it here or go to all visualizations to add other panel types
Queries 1
Transformations 0
Explain
Run queries
Builder
Code
Metric
Label filters
process_cpu_seconds_total
Select label
Select value
Operations
hint: add rate
process_cpu_seconds_total


Table view
Last 6 hours
Refresh
New panel
Suggestions
All visualizations
Search for...
Time series
Stat
{__name__="process_cpu_seconds_total", app="prometheus", instance="localhost:9090", job="prometheus"}
Queries 1
Transformations 0
Data source
prometheus
Query o...
MD = auto = 500
Interval = 30s
Query inspector
A
(prometheus)
Explain
Run queries
Builder
Code
Metric
Label filters
process_cpu_seconds_total
Select label
Select value


Queries 1
Transformations 0
Data source
prometheus
Query o...
MD = auto = 500
Interval = 30s
Query inspector
A
(prometheus)
Explain
Run queries
Builder
Code
Metric
Label filters
go_memstats_alloc_bytes
Select label
Select value


Table view
Last 6 hours
Refresh
New panel
Suggestions
All visualizations
Search for...
Time series
Stat
{__name__="go_memstats_alloc_bytes", app="prometheus", instance="localhost:9090", job="prometheus"}
Queries 1
Transformations 0
Data source
prometheus
Query o...
MD = auto = 500
Interval = 30s
Query inspector
A
(prometheus)
Explain
Run queries
Builder
Code
Metric
Label filters
go_memstats_alloc_bytes
Select label
Select value

SRI ESHWAR COLLEGE OF ENGINEERING DEPARTMENT OF CSE		
PARAMETERS	**MAX MARKS**	**MARKS OBTAINED**
Pre Lab	20	
Program	25	
Output	20	
Viva	10	
Total	75	
SIGNATURE OF THE FACULTY		

Result

Grafana was successfully deployed and integrated with Prometheus as a data source. The monitoring metrics collected by Prometheus were visualized using Grafana dashboards, enabling real-time monitoring and analysis of system performance.

Ex.No: 10	Install and Configure the ELK Stack to Collect and Visualize Application Logs
Date:	

Aim

To install and configure the ELK Stack (Elasticsearch, Logstash, and Kibana) for collecting, storing, and visualizing application logs.

Prerequisites

- Windows/Linux System
- Java Installed
- Administrator Privileges

Procedure

Step 1: Download Elasticsearch

1. Open a browser and Visit the Elasticsearch download page.

   https://www.elastic.co/downloads/elasticsearch
2. Download the Windows ZIP package.
3. Extract the ZIP file.

Step 2: Start Elasticsearch

1. Open the extracted Elasticsearch folder.
2. Navigate to:

   bin
3. Run:

   elasticsearch.bat
4. Verify Elasticsearch is running by opening:

   https://localhost:9200

Username: elastic
Password: M-6=pm3PDbybOaE3Hkg8
Expected Output:

```
{
  "name": "node-1",
  "cluster_name": "elasticsearch",
  "version": {
    "number": "9.x"
  }
}
```

Step 3: Download Kibana

1. Visit the Kibana download page.

   https://www.elastic.co/downloads/kibana
2. Download the Windows ZIP package.

3. Extract the ZIP file.

Step 4: Start Kibana

1. Open the Kibana folder.
2. Navigate to:

   bin
3. Run:

   kibana.bat
4. Open:

   http://localhost:5601

Kibana dashboard should load successfully.

Step 5: Download Logstash

1. Visit the Logstash download page.

   https://www.elastic.co/downloads/logstash
2. Download the Windows ZIP package.
3. Extract the ZIP file.
4. Create Pipeline Configuration:
   D:\logstash-9.5.3\config\pipeline.conf

Open Notepad and create:

Paste the code:

```
input {
  stdin { }
}

output {
  stdout {
    codec => rubydebug
  }
}
```

Save the file.

5. Configure pipelines.yml

Open:

D:\logstash-9.5.3\config\pipelines.yml

Replace its contents with:

```
- pipeline.id: main
  path.config: "D:/logstash-9.5.3/config/pipeline.conf"
```

Save the file.

Step 6: Start Logstash

Now run Logstash **using that file explicitly**:

```
cd D:\logstash-9.5.3
```

.\bin\logstash.bat -f D:\logstash-9.5.3\config\pipeline.conf

Step 7: Create Index Pattern in Kibana

1. Open Kibana.
2. Navigate to **Management → Data Views**.
3. Click **Create Data View**.
4. Enter:

   application-logs*

5. Save the Data View.

Step 8: Visualize Logs

1. Open **Discover** in Kibana.
2. Select:

   application-logs*

3. View the collected log entries.
4. Create charts and visualizations if required.

Output

Elasticsearch started successfully on port 9200.
Kibana started successfully on port 5601.

Logstash processed application logs successfully.

Logs stored in Elasticsearch index:
application-logs

Logs visualized successfully in Kibana dashboard.

SRI ESHWAR COLLEGE OF ENGINEERING DEPARTMENT OF CSE		
PARAMETERS	**MAX MARKS**	**MARKS OBTAINED**
Pre Lab	20	
Program	25	
Output	20	
Viva	10	
Total	75	
SIGNATURE OF THE FACULTY		

Result

The ELK Stack (Elasticsearch, Logstash, and Kibana) was successfully installed and configured. Application logs were collected using Logstash, stored in Elasticsearch, and visualized through Kibana dashboards.